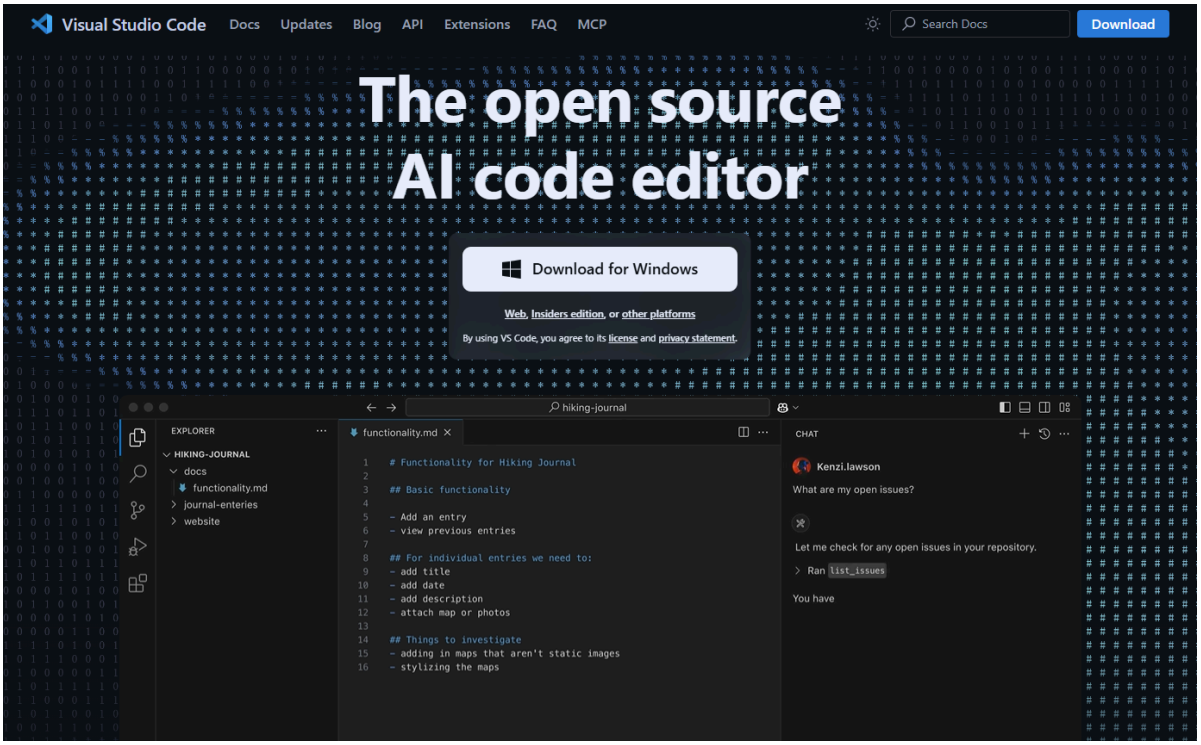
3. Setting Up the ROS 2 IDE

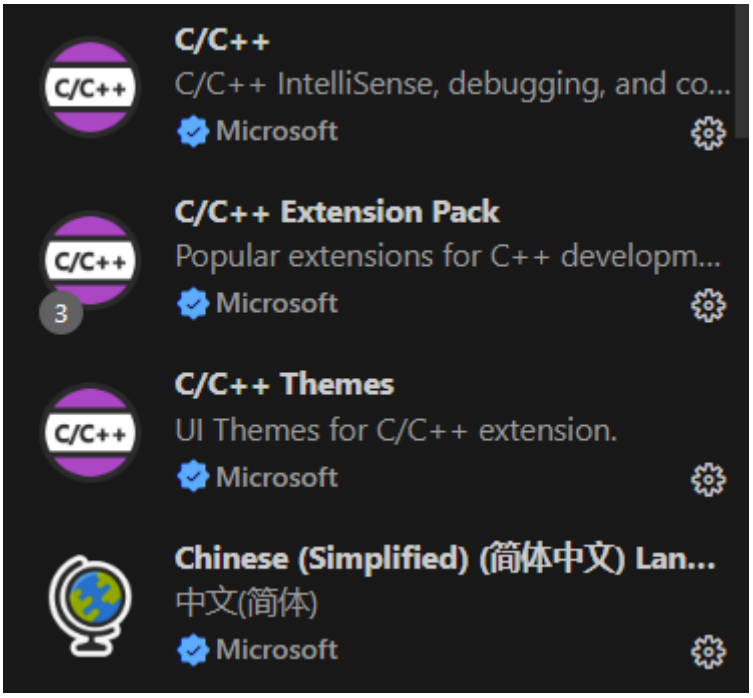
In theory, in ROS 2, you can write basic ROS 2 programs with just a notepad, but to improve development efficiency, you can install the integrated development environment VSCode.

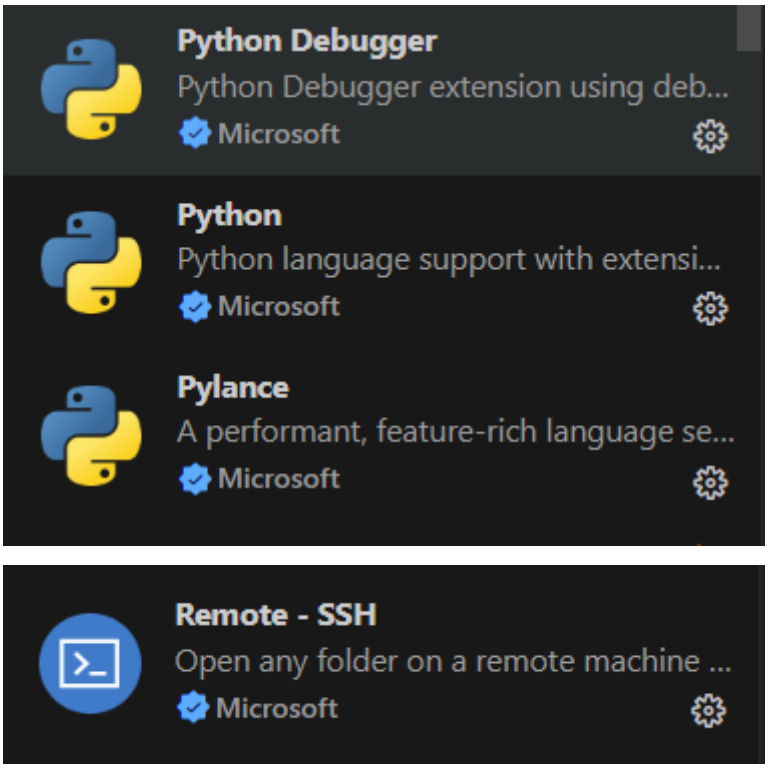
1. Using the VSCode Development Environment

- Search for VSCode in a browser, select the installation package for the corresponding system platform, and download and install it.



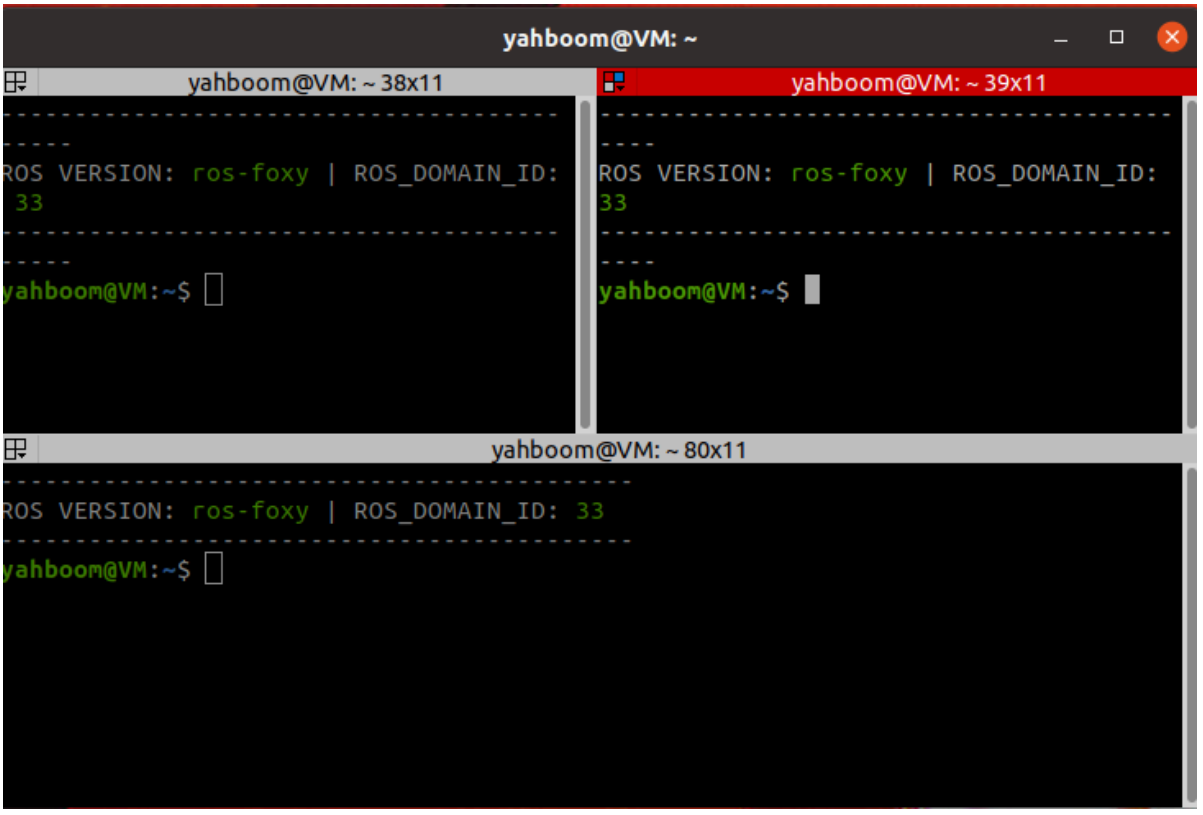
- Install commonly used plug-ins to help improve work efficiency.





2. Using the Terminal

In ROS 2, the terminal is used frequently. A relatively easy-to-use terminal is recommended: Terminator. The effect is as follows:



2.1. Installation

```
sudo apt install terminator
```

2.2. Launching

```
Use the shortcut `Ctrl+Alt+T` to launch
```

2.3. Common Terminator Shortcuts

For operations within the same tab:

```
Alt+Up                          // Move to the terminal above
Alt+Down                        // Move to the terminal below
Alt+Left                        // Move to the terminal on the left
Alt+Right                       // Move to the terminal on the right
Ctrl+Shift+O                    // Split the terminal horizontally
Ctrl+Shift+E                    // Split the terminal vertically
Ctrl+Shift+Right                // Move the split bar to the right in a
vertically split terminal
Ctrl+Shift+Left                 // Move the split bar to the left in a vertically
split terminal
Ctrl+Shift+Up                   // Move the split bar up in a horizontally split
terminal
Ctrl+Shift+Down                 // Move the split bar down in a horizontally
split terminal
Ctrl+Shift+S                    // Hide/show the scroll bar
Ctrl+Shift+F                    // Search
Ctrl+Shift+C                    // Copy the selected content to the clipboard
Ctrl+Shift+V                    // Paste the clipboard content here
Ctrl+Shift+W                    // Close the current terminal
Ctrl+Shift+Q                    // Exit the current window; all terminals in the
current window will be closed
Ctrl+Shift+X                    // Maximize the current terminal
Ctrl+Shift+Z                    // Maximize the current terminal and enlarge the
font
Ctrl+Shift+N or Ctrl+Tab        // Move to the next terminal
Ctrl+Shift+P or Ctrl+Shift+Tab  // Crtl+Shift+Tab moves to the previous terminal
```

For operations between tabs:

```
F11                             // Toggle full screen
Ctrl+Shift+T                    // Open a new tab
Ctrl+PageDown                   // Move to the next tab
Ctrl+PageUp                     // Move to the previous tab
Ctrl+Shift+PageDown             // Swap the current tab with the tab after it
Ctrl+Shift+PageUp               // Swap the current tab with the tab before it
Ctrl+Plus (+)                   // Increase the font size
Ctrl+Minus (-)                  // Decrease the font size
Ctrl+Zero (0)                   // Restore the font to the original size
Ctrl+Shift+R                    // Reset the terminal state
Ctrl+Shift+G                    // Reset the terminal state and clear the screen
Super+g                         // Bind all terminals so that input to one can be
input to all
Super+Shift+G                   // Unbind
Super+t                         // Bind all terminals of the current tab so that
input to one terminal is automatically input to the others
Super+Shift+T                   // Unbind
```

```
Ctrl+Shift+I                    // Open a window; the new window uses the same
process as the original window
Super+i                         // Open a new window; the new window uses a
different process from the original window
```

3. Using Git

3.1. Installation

In daily work, since everything involves team collaboration and version management, Git is an unavoidable skill. Git is a free and open source distributed version control system. To install Git under Ubuntu:

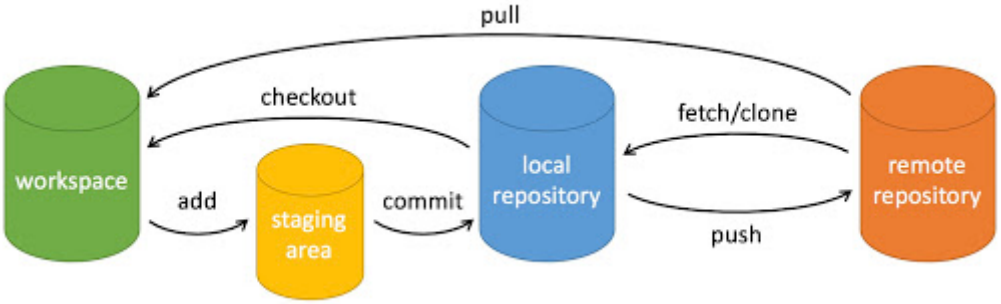
```
sudo apt install git
```

3.2. Basic Git Operations

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

This chapter introduces the commands for creating and committing snapshots of your project.

The commonly used Git commands are the following 6: **git clone**, **git push**, **git add**, **git commit**, **git checkout**, **git pull**. We will introduce them in detail later.



Description:

- workspace: Working area
- staging area: Staging area/cache area
- local repository: Repository or local repository
- remote repository: Remote repository

A simple operation procedure:

```
$ git init
$ git add .
$ git commit
```

- git init - Initializes a repository.
- git add . - Adds files to the staging area.
- git commit - Adds the contents of the staging area to the repository.

3.2.1. Repository Creation Commands

The following table lists the Git commands for creating a repository:

Command	Description
`git init`	Initializes a repository
`git clone`	Copies a remote repository, that is, downloads a project.

3.2.2. Committing and Modifying

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

The following table lists the commands for creating and committing snapshots of your project:

Command	Description
`git add`	Adds files to the staging area
`git status`	Views the current state of the repository and shows the files that have changed.
`git diff`	Compares differences between files, that is, the differences between the staging area and the working area.
`git commit`	Commits the staging area to the local repository.
`git reset`	Rolls back to a previous version.
`git rm`	Deletes files from the staging area and the working area.
`git mv`	Moves or renames files in the working area.
`git checkout`	Switches branches.
`git switch` (introduced in Git 2.23)	Switches branches more clearly.
`git restore` (introduced in Git 2.23)	Restores or undoes changes to files.

3.2.3. Commit Log

Command	Description
`git log`	Views the historical commit records
`git blame <file>`	Views the historical modification records of a specified file in list form

3.2.4. Remote Operations

Command	Description
`git remote`	Remote repository operations
`git fetch`	Retrieves code from the remote repository
`git pull`	Downloads remote code and merges it
`git push`	Uploads remote code and merges it

For more information about using Git, enter `git --help` in the terminal to view the help documentation.